



Sensores e Sinais

Aula Prática: Transformada de Z e Filtros Digitais

Paulo Pedreiras, Pedro Fonseca, Luis Silva, DETI/UA/IT

2025-2026

Informações da Aula

- **Duração:** 2 horas em sala + 2 horas de trabalho extra-aula
- **Tópico:** Transformada de Z e Filtros Digitais
- **Baseado em:** Oppenheim et al, Capítulo 10, e Smith et al, Capítulos 14 e 33
- **Preparação prévia:** Aula TP relativa à Transformada de Z

Objetivos de Aprendizagem

- Compreender a Transformada de Z
- Saber calcular a transformada de Z e a ROC de alguns sinais elementares
- Saber analisar a resposta de sistemas usando a Transformada de Z, incluindo filtros digitais
- Saber usar ferramentas computacionais (Octave/Matlab) no contexto de análise e síntese de sistemas com Transformadas de Z

1 Parte 1: Exercícios em aula (1h45m)

1.1 Exercício 1.1: Cálculo Básico da Transformada Z

Cálculo Manual:

Calcule a Transformada Z e a Região de Convergência (ROC) para as seguintes sequências:

a) $x[n] = (0.5)^n u[n]$

b) $x[n] = -(0.8)^n u[-n - 1]$

c) $x[n] = \delta[n] + 2\delta[n - 1] + \delta[n - 2]$

d) $x[n] = n(0.99)^n u[n]$ (aplicar propriedade, slide TP 11, exp. 10.5.8)

Verificação em Octave:

O Octave possui um package denominado “symbolic” que é uma ferramenta extremamente útil para vários contextos, incluindo o estudo e aplicação da Transformada Z.

Este package permite realizar cálculos algébricos exatos, em contraste com os cálculos numéricos tradicionais, o que é particularmente valioso quando se trabalha com expressões matemáticas complexas.

No caso concreto da Transformada Z, o package simbólico facilita várias tarefas importantes:

- Permite calcular transformadas Z de forma analítica, manipulando expressões como variáveis simbólicas em vez de valores numéricos, o que ajuda a compreender melhor as relações matemáticas subjacentes;
- Torna mais simples a expansão em frações parciais, uma técnica fundamental para calcular transformadas Z inversas, pois o package pode fatorizar polinômios e decompor funções racionais automaticamente (ver função “[r, p, k] = residue(b, a)”);
- Permite verificar cálculos manuais de forma rápida. Os estudantes podem resolver problemas analiticamente no papel e depois confirmar os resultados usando o Octave, o que reforça a aprendizagem e reduz erros;
- A capacidade de simplificar expressões algébricas complexas também é crucial quando se trabalha com funções de transferência que envolvem múltiplos polos e zeros.

O script seguinte ilustra o uso deste package. Como trabalho para casa devem completar o script para verificar os cálculos das restantes alíneas deste exercício.

Nota: o package “symbolic” pode não estar instalado por defeito. Se for esse o caso, digite o comando **pkg install -forge symbolic** na janela de comandos do Octave. O package “symbolic” recorre ao python e necessita do package python “sympy”. Se o script assinala este erro deve abrir um terminal no Linux e digitar o comando **pip install sympy**.

```
1 % Exercício 1.1 - Verificacao usando computacao numerica e simbolica
2 pkg load symbolic
3 clear
4
5 % Ex 1.1 a): x[n] = (0.5)^n u[n]
6 % Verificacao numerica
7 n = 0:20;
8 x_a = (0.5).^n;
9 figure(1);
10 stem(n, x_a);
11 title('x[n] = (0.5)^n u[n]');
12 xlabel('n'); ylabel('x[n]');
13 grid on;
14
15 % Verificacao simbolica
16 syms n;
17 f=(0.5)^n;
18 F=ztrans(f);
19 display('Transformada de Z, Ex.1.1 a)');
20 display(F)
21
```

```

22 % Ex 1_3 a): x[n] = delta[n] + 2*delta[n-1] + delta[n-2]
23 % Verificacao numerica
24 % ----
25 % ----
26
27 % Verificacao simbolica
28 syms n;
29 f=kroneckerDelta(n,0)+2*kroneckerDelta(n,1)+kroneckerDelta(n,2);
30 F=ztrans(f);
31 display('Transformada de Z, Ex.1.1 c)');
32 display(F)

```

Listing 1: Exercício 1.1 - Verificação

1.2 Exercício 1.2: Transformada Z Inversa

Cálculo Manual:

Determine a transformada Z inversa usando expansão em frações parciais:

a) $X(z) = \frac{2z}{(z-0.5)(z-0.3)}, |z| > 0,5$

b) $X(z) = \frac{z^2}{z^2-0.5z-0.5}, |z| > 1$

Nota: quando a expressão da transformada de Z está expressa com potências positivas de z, em geral é melhor efectuar a expansão em fracções parciais sobre a expressão dividida por z, e voltar a multiplicar por z no fim. Isso permite obter directamente expressões na forma $\frac{z}{z-a}$, directamente invertíveis, sem cálculos adicionais (normalmente “time-shifts”).

Implementação em Octave:

```

1 % Exercicio 1.2 - Transformada Z Inversa
2 % Parte (a)
3 disp("a)")
4 num = [2]; % 0 numerador correcto e [2 0]. Neste caso fez-se divisao
   por z para simplificar a inversa
5 den = conv([1 -0.5], [1 -0.3]);
6 [r, p, k] = residue(num, den);
7
8 disp('Coeficientes da Fracao Parcial (dividida por z):');
9 disp('Residuos:'); disp(r);
10 disp('Polos:'); disp(p);
11 disp('Termo direto:'); disp(k);
12 disp('Nao esquecer de multiplicar por z a fracao final!')
13
14 % Reconstruir a sequencia
15 n = 0:20;
16 x_n = r(1)*(p(1).^n) + r(2)*(p(2).^n);
17 figure(121);
18 stem(n, x_n);
19 title('Transformada Z Inversa de X(z) Ex 1.2 a)');
20 xlabel('n'); ylabel('x[n]');
21 grid on;

```

Listing 2: Exercício 1.2 - Transformada Z Inversa

1.3 Exercício 1.3: Análise de Sistemas Usando a Transformada Z

Enunciado do Problema:

Um sistema LTI em tempo discreto é descrito pela equação diferença:

$$y[n] - 0.8y[n-1] = x[n] + 0.5x[n-1] \quad (1)$$

- a) Determine a função de transferência $H(z)$
- b) Determine os polos e zeros
- c) Trace o diagrama de polos e zeros
- d) Calcule as primeiras 5 amostras da resposta impulsional ($h[n], n = 0 \dots 4$), quer partindo da equação diferença inicial quer da transformada inversa de $H(z)$. Compare ambos os resultados.
- e) O sistema é estável?

Implementação em Octave:

```

1 % Exercício 1.3 - Analise de Sistemas
2 % Coeficientes da funcao de transferencia
3 b = [____]; % Numerador (zeros)
4 a = [____]; % Denominador (polos)
5
6 % (b) Encontrar polos e zeros
7 zeros_H = roots(b);
8 poles_H = roots(a);
9
10 disp('Zeros:'); disp(zeros_H);
11 disp('Polos:'); disp(poles_H);
12
13 % (c) Diagrama de Polos e Zeros
14 figure(3);
15 zplane(b, a);
16 title('Diagrama de Polos e Zeros');
17 grid on;
18
19 % Adicionar circulo unitario para verificacao de estabilidade
20 hold on;
21 theta = linspace(0, 2*pi, 100);
22 plot(cos(theta), sin(theta), 'r--', 'LineWidth', 1.5);
23 hold off;
24
25 % (d) Resposta ao Impulso
26 N = 40;
27 impulse = [1 zeros(1, N-1)];
28 h = filter(b, a, impulse);
29
30 figure(4);
31 stem(0:N-1, h);
32 title('Resposta ao Impulso h[n]');
33 xlabel('n'); ylabel('h[n]');
34 grid on;
35
36 % (e) Verificacao de estabilidade
37 if all(abs(poles_H) < 1)

```

```

38     disp('Sistema e ESTAVEL (todos os polos dentro do circulo unitario)')
39     );
40 else
41     disp('Sistema e INSTAVEL');
42 end
43 % Resposta em Frequencia
44 [H, w] = freqz(b, a, 512);
45 figure(5);
46 subplot(2,1,1);
47 plot(w/pi, 20*log10(abs(H)));
48 title('Resposta em Magnitude');
49 xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
50 ylabel('Magnitude (dB)');
51 grid on;
52
53 subplot(2,1,2);
54 plot(w/pi, angle(H)*180/pi);
55 title('Resposta em Fase');
56 xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
57 ylabel('Fase (graus)');
58 grid on;

```

Listing 3: Exercício 1.3 - Análise de Sistemas

1.4 Exercício 1.4: Projeto de Filtro Digital Simples

Enunciado do Problema:

Projete e implemente um filtro de média móvel simples com 5 coeficientes:

$$y[n] = \frac{x[n] + x[n-1] + x[n-2] + x[n-3] + x[n-4]}{5} \quad (2)$$

- Determine a função de transferência $H(z)$
- Determine os polos e zeros
- Analise a resposta em frequência
- Teste o filtro com um “sinal ruidoso” (*).
Use uma senoide com amplitude unitária como sinal, e adicione ruído com uma variância de 0.25, que é “elevado” e bem visível.
- Repita os passos c) e d) com usando um filtro de 3 coeficientes e de 10 coeficientes. Compare os resultados.

As alíneas a) e b) devem ser feitas manualmente. As restantes alíneas devem ser feitas em Octave.

(*) Muitos sensores reais apresentam na sua saída sinais que estão afectados por ruído e interferência. Em muitos casos (especialmente processos físicos) o ruído pode modelar-se com um grau razoável de aproximação por um sinal de média nula e com uma distribuição gaussiana (ou normal). A função do Octave “randn(M, N)” gera uma matriz com M*N amostras e variância unitária, podendo ser usada para modelar a presença de ruído, somando-se algebricamente ao sinal “puro”.

Se se pretender um sinal com variância não unitária, deve multiplicar-se o vector gerado pelo `randn()` (com variância unitária) pela raiz quadrada da variância desejada.

Implementação em Octave:

```
1 % Exercício 1.4 - Filtro de Media Movel
2
3 % (a) Funcao de transferencia
4 N_taps = 5;
5 b = ones(1, N_taps) / N_taps;
6 a = 1;
7
8 disp('Coeficientes da Funcao de Transferencia:');
9 disp('Numerador b:'); disp(b);
10 disp('Denominador a:'); disp(a);
11
12 % (b) Polos e Zeros
13 zeros_filter = roots(b);
14 disp('Zeros do Filtro:'); disp(zeros_filter);
15 disp('Polos do Filtro: z = 0 (na origem)');
16
17 figure(6);
18 zplane(b, a);
19 title('Filtro de Media Movel - Diagrama de Polos e Zeros');
20
21 % (c) Resposta em Frequencia
22 [H, w] = freqz(b, a, 512);
23 figure(7);
24 subplot(2,1,1);
25 plot(w/pi, abs(H));
26 title('Filtro de Media Movel - Resposta em Magnitude');
27 xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
28 ylabel('Magnitude');
29 grid on;
30
31 subplot(2,1,2);
32 plot(w/pi, angle(H));
33 title('Resposta em Fase');
34 xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
35 ylabel('Fase (rad)');
36 grid on;
```

Listing 4: Exercício 1.4 - Filtro de Média Móvel

```
1 % (d) Teste com sinal ruidoso
2 n = 0:199;
3 fs = 1000; % Frequencia de amostragem (Hz)
4 f_signal = 50; % Frequencia do sinal (Hz)
5 clean_signal = sin(______);
6 noise = _____;
7 noisy_signal = clean_signal + noise;
8
9 % Aplicar filtro
10 filtered_signal = filter(b, a, noisy_signal);
11
12 figure(8);
13 subplot(3,1,1);
14 plot(n, clean_signal);
15 title('Sinal Original Limpo');
16 xlabel('Amostra n'); ylabel('Amplitude');
17 grid on;
18
19 subplot(3,1,2);
20 plot(n, noisy_signal);
21 title('Sinal Ruidoso');
22 xlabel('Amostra n'); ylabel('Amplitude');
23 grid on;
24
25 subplot(3,1,3);
26 plot(n, filtered_signal);
27 title('Sinal Filtrado');
28 xlabel('Amostra n'); ylabel('Amplitude');
29 grid on;
```

Listing 5: Exercício 1.4 - Teste com Sinal Ruidoso (continuação)

2 PARTE 2: Exercícios complementares, para resolver fora da aula

2.1 Exercício 2.1: Propriedades da Transformada Z

Cálculo Manual:

Dado $x[n] = (0,6)^n u[n]$ com transformada Z $X(z) = \frac{z}{z-0,6}$, $|z| > 0,6$

Use as propriedades da transformada Z para determinar:

- a) $\mathcal{Z}\{x[n-2]\}$ (Deslocamento no tempo)
- b) $\mathcal{Z}\{n \cdot x[n]\}$ (Multiplicação por n)
- c) $\mathcal{Z}\{(0,5)^n x[n]\}$ (Multiplicação exponencial)
- d) $\mathcal{Z}\{x[n] * x[n]\}$ onde $*$ denota convolução

Verificação em Octave:

```

1  % Exercício 2.1 - Verificacao
2  n = 0:30;
3  x_n = (0.6).^n;
4
5  % (a) Deslocamento no tempo: x[n-2]
6  x_shifted = [0 0 x_n(1:end-2)];
7
8  % (b) Multiplicacao por n
9  x_mult_n = n .* x_n;
10
11 % (c) Multiplicacao exponencial
12 x_exp = (0.5).^n .* x_n;
13
14 % (d) Convolucao
15 x_conv = conv(x_n, x_n);
16
17 figure(9);
18 subplot(2,2,1);
19 stem(n, x_shifted);
20 title('x[n-2]');
21 xlabel('n'); ylabel('Amplitude');
22 grid on;
23
24 subplot(2,2,2);
25 stem(n, x_mult_n);
26 title('n \cdot x[n]');
27 xlabel('n'); ylabel('Amplitude');
28 grid on;
29
30 subplot(2,2,3);
31 stem(n, x_exp);
32 title('(0.5)^n \cdot x[n]');
33 xlabel('n'); ylabel('Amplitude');
34 grid on;
35
36 subplot(2,2,4);
37 stem(0:length(x_conv)-1, x_conv);
38 title('x[n] * x[n]');

```



```
39 xlabel('n'); ylabel('Amplitude');  
40 grid on;
```

Listing 6: Exercício 2.1 - Verificação

2.2 Exercício 2.2: Análise de Sistema de Segunda Ordem

Enunciado do Problema:

Um filtro digital de segunda ordem tem a função de transferência:

$$H(z) = \frac{0,05z^2}{z^2 - 1,5z + 0,9} \quad (3)$$

- Determine a equação diferença
- Calcule os polos e zeros
- Verifique a estabilidade
- Trace a resposta ao degrau
- Trace a resposta em frequência e identifique o tipo de filtro (passa-baixo, passa-alto, passa-banda, rejeita-banda)

Cálculo Manual:

Converta $H(z)$ para a forma de equação às diferenças através de multiplicação cruzada.

Implementação em Octave:

```

1 % Exercício 2.2 - Sistema de Segunda Ordem
2
3 % Funcao de transferencia
4 num = [0.05 0 0];
5 den = [1 -1.5 0.9];
6
7 % (a) Mostrar equacao as diferencas
8 disp('Equacao as Diferencas:');
9 disp('y[n] - 1.5*y[n-1] + 0.9*y[n-2] = 0.05*x[n]');
10
11 % (b) Polos e zeros
12 z_zeros = roots(num);
13 z_poles = roots(den);
14
15 disp('Zeros:'); disp(z_zeros);
16 disp('Polos:'); disp(z_poles);
17
18 % (c) Estabilidade
19 figure(10);
20 zplane(num, den);
21 title('Filtro de Segunda Ordem - Diagrama de Polos e Zeros');
22
23 pole_magnitudes = abs(z_poles);
24 disp('Magnitudes dos Polos:'); disp(pole_magnitudes);
25
26 if all(pole_magnitudes < 1)
27     disp('Sistema e ESTAVEL');
28 else
29     disp('Sistema e INSTAVEL');
30 end
31
32 % (d) Resposta ao Degrau
33 N = 50;

```

```
34 step_input = ones(1, N);  
35 step_output = filter(num, den, step_input);  
36  
37 figure(11);  
38 stem(0:N-1, step_output);  
39 title('Resposta ao Degrau');  
40 xlabel('n'); ylabel('y[n]');  
41 grid on;
```

Listing 7: Exercício 2.2 - Sistema de Segunda Ordem

```
1 % (e) Resposta em Frequencia
2 [H, w] = freqz(num, den, 512);
3
4 figure(12);
5 subplot(2,1,1);
6 plot(w/pi, 20*log10(abs(H)));
7 title('Resposta em Frequencia - Magnitude');
8 xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
9 ylabel('Magnitude (dB)');
10 grid on;
11
12 subplot(2,1,2);
13 plot(w/pi, unwrap(angle(H))*180/pi);
14 title('Resposta em Frequencia - Fase');
15 xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
16 ylabel('Fase (graus)');
17 grid on;
18
19 % Identificar tipo de filtro
20 [max_val, max_idx] = max(abs(H));
21 max_freq = w(max_idx);
22
23 if max_freq < pi/4
24     disp('Tipo de Filtro: PASSA-BAIXO');
25 elseif max_freq > 3*pi/4
26     disp('Tipo de Filtro: PASSA-ALTO');
27 else
28     disp('Tipo de Filtro: PASSA-BANDA');
29 end
```

Listing 8: Exercício 2.2 - Resposta em Frequência (continuação)

2.3 Exercício 2.3: Projeto de Filtro Notch Simples

Enunciado do Problema:

Projete um filtro notch para remover uma interferência de 60 Hz de um sinal amostrado a 500 Hz.

Um filtro notch pode ser projetado usando a função de transferência:

$$H(z) = \frac{z^2 - 2\cos(\omega_0)z + 1}{z^2 - 2r \cdot \cos(\omega_0)z + r^2} \quad (4)$$

onde ω_0 é a frequência normalizada do notch e r controla a largura do notch (tipicamente $r = 0,9-0,99$).

- a) Calcule ω_0 para 60 Hz com $f_s = 500$ Hz
- b) Projete o filtro com $r = 0,95$
- c) Determine os coeficientes da função de transferência
- d) Trace a resposta em frequência
- e) Teste com um sinal composto contendo interferência de 60 Hz

Cálculo Manual:

Calcule $\omega_0 = 2\pi f_0/f_s$ e expanda os polinômios da função de transferência.

Implementação em Octave:

```

1 % Exercício 2.3 - Projeto de Filtro Notch
2
3 % (a) Calcular frequencia normalizada
4 fs = 500;           % Frequencia de amostragem (Hz)
5 f_notch = 60;       % Frequencia do notch (Hz)
6 omega_0 = 2*pi*f_notch/fs;
7
8 disp(['Frequencia normalizada do notch: ', num2str(omega_0), ' rad/
9 amostra']);
10 disp(['Frequencia normalizada (x pi): ', num2str(omega_0/pi)]);
11
12 % (b) & (c) Projetar filtro
13 r = 0.95;           % Parametro de largura do notch
14
15 % Coeficientes do numerador (zeros)
16 b = [1 -2*cos(omega_0) 1];
17
18 % Coeficientes do denominador (polos)
19 a = [1 -2*r*cos(omega_0) r^2];
20
21 disp('Coeficientes da Funcao de Transferencia:');
22 disp('Numerador (b):'); disp(b);
23 disp('Denominador (a):'); disp(a);
24
25 % Polos e zeros
26 z_zeros = roots(b);
27 z_poles = roots(a);
28
29 disp('Zeros do Filtro:'); disp(z_zeros);
30 disp('Polos do Filtro:'); disp(z_poles);
31
32 % Diagrama de polos e zeros
33 figure(13);
34 zplane(b, a);
35 title('Filtro Notch - Diagrama de Polos e Zeros');
36 hold on;
37 theta = linspace(0, 2*pi, 100);
38 plot(cos(theta), sin(theta), 'r--', 'LineWidth', 1.5);
39 hold off;
40 grid on;
41
42 % (d) Resposta em Frequencia
43 [H, w] = freqz(b, a, 512);
44
45 figure(14);
46 subplot(2,1,1);
47 plot(w/pi, 20*log10(abs(H)));
48 title('Filtro Notch - Resposta em Magnitude');
49 xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
50 ylabel('Magnitude (dB)');
51 grid on;
52 hold on;
53 % Marcar frequencia do notch
54 plot(omega_0/pi, 20*log10(abs(freqz(b, a, omega_0))), 'ro', 'MarkerSize',
55      10);
56 legend('Resposta', 'Frequencia do Notch');

```

```
55 hold off;
56
57 subplot(2,1,2);
58 plot(w/pi, unwrap(angle(H))*180/pi);
59 title('Filtro Notch - Resposta em Fase');
60 xlabel('Frequencia Normalizada ( $\times \pi$  rad/amostra)');
61 ylabel('Fase (graus)');
62 grid on;
```

Listing 9: Exercício 2.3 - Projeto de Filtro Notch

```

1 % (e) Teste com sinal composto
2 n = 0:999;          % 1000 amostras
3 t = n/fs;           % Vetor de tempo
4
5 % Sinal desejado: 20 Hz
6 f_desired = 20;
7 signal_desired = sin(2*pi*f_desired*t);
8
9 % Interferencia: 60 Hz
10 interference = 0.8*sin(2*pi*f_notch*t);
11
12 % Sinal composto (desejado + interferencia)
13 signal_composite = signal_desired + interference;
14
15 % Aplicar filtro notch
16 signal_filtered = filter(b, a, signal_composite);
17
18 % Visualizacao
19 figure(15);
20 subplot(4,1,1);
21 plot(t(1:200), signal_desired(1:200));
22 title('Sinal Desejado (20 Hz)');
23 xlabel('Tempo (s)'); ylabel('Amplitude');
24 grid on;
25
26 subplot(4,1,2);
27 plot(t(1:200), interference(1:200));
28 title('Interferencia (60 Hz)');
29 xlabel('Tempo (s)'); ylabel('Amplitude');
30 grid on;
31
32 subplot(4,1,3);
33 plot(t(1:200), signal_composite(1:200));
34 title('Sinal Composto (Desejado + Interferencia)');
35 xlabel('Tempo (s)'); ylabel('Amplitude');
36 grid on;
37
38 subplot(4,1,4);
39 plot(t(1:200), signal_filtered(1:200));
40 title('Sinal Filtrado (apos Filtro Notch)');
41 xlabel('Tempo (s)'); ylabel('Amplitude');
42 grid on;
43
44 % Analise espectral
45 figure(16);
46 N_fft = 1024;
47 f_axis = (0:N_fft-1)*fs/N_fft;
48
49 % Espectro do sinal composto
50 X_composite = abs(fft(signal_composite, N_fft));
51 subplot(2,1,1);
52 plot(f_axis(1:N_fft/2), 20*log10(X_composite(1:N_fft/2)));
53 title('Espectro do Sinal Composto');
54 xlabel('Frequencia (Hz)'); ylabel('Magnitude (dB)');
55 grid on;
56 hold on;
57 plot([f_notch f_notch], ylim, 'r--', 'LineWidth', 2);

```



```
58 legend('Espectro', 'Frequencia de Interferencia (60 Hz)');
59 hold off;
60
61 % Espectro do sinal filtrado
62 X_filtered = abs(fft(signal_filtered, N_fft));
63 subplot(2,1,2);
64 plot(f_axis(1:N_fft/2), 20*log10(X_filtered(1:N_fft/2)));
65 title('Espectro do Sinal Filtrado');
66 xlabel('Frequencia (Hz)'); ylabel('Magnitude (dB)');
67 grid on;
68 hold on;
69 plot([f_notch f_notch], ylim, 'r--', 'LineWidth', 2);
70 legend('Espectro', 'Frequencia Removida (60 Hz)');
71 hold off;
72
73 % Calcular atenuacao na frequencia de interferencia
74 H_at_notch = freqz(b, a, omega_0);
75 attenuation_dB = 20*log10(abs(H_at_notch));
76 disp(['Atenuacao em ', num2str(f_notch), ' Hz: ', ...
77       num2str(attenuation_dB), ' dB']);
```

Listing 10: Exercício 2.3 - Teste com Sinal Composto (continuação)